

RAG over a Financial Filing

Approach, Design Decisions and Findings

Document	Apple Inc. Form 10-Q, quarter ended June 25, 2022 (28 pages)
Repository	Smart_Data_Sol_RAG
Result	19 of 20 on a fixed evaluation set

1. Summary

A question-answering system over a 28-page SEC filing, covering the three content types the brief asks for: text, tables and figures.

Two findings shaped everything:

The hard part is reading tables, not retrieving them. Every income statement in this filing has four value columns. A conventional pipeline gives answers that are fluent, correctly cited, and numerically wrong, because the column headers are lost during extraction and the model cannot tell a quarterly figure from a year-to-date one.

The document has no figures. One embedded image in 28 pages: a 46x56 pixel logo. The figure pipeline is built and runs; it reports their absence rather than inventing charts.

2. Approach

2.1 Measure the document first

Before writing a pipeline, the PDF was inspected:

Property	Value
Pages	28
Tables	30
Pages with narrative text	28
Embedded images	1 (46x56 px, page 1)

This changed the plan. "Text, tables, figures" implies a document with charts. A 10-Q is mostly financial tables and prose, with no analytical graphics at all. Effort went to table fidelity instead of chart extraction.

2.2 Pipeline

```
PDF
|
| pdfplumber -> tables (and their positions on the page)
| PyMuPDF    -> images
| page text extracted with table areas removed
v
elements: 28 text, 30 tables, 1 figure
|
| each table -> an LLM-written description
| each image -> a vision-model caption
v
two stores, joined by one id
|
```

```

+-- Chroma : embedding of the DESCRIPTION -> searched
+-- SQLite : the VERBATIM content         -> returned
|
v
question -> vector -> ids -> rows -> answer with page citations

```

Stack. Python 3.13, pdfplumber, PyMuPDF, `gemini-3.5-flash-lite`, `gemini-embedding-001`, ChromaDB, SQLite, Streamlit.

The answering step was originally on the stronger `gemini-3.6-flash`. It moved to the lite model only because that model's free-tier quota ran out during evaluation. It is one line in `config.yaml`, so the results below are a floor, not a ceiling.

2.3 The core idea: search the description, return the original

Embedding a raw financial table does not work. Its text is essentially

```
iPhone 40,665 39,570 162,863 153,105
```

Digits, with no meaning attached. Someone asking *"how did iPhone revenue compare to last year?"* uses none of those words, so similarity search ranks the table low and never retrieves it. The system then answers from a narrative paragraph that merely mentions iPhone, and produces a confident wrong number.

So each table and figure gets a written description:

> *"Net sales by product category for the three and nine months ended June 25 > 2022 versus June 26 2021, covering iPhone, Mac, iPad, Wearables and Services."*

That is what gets embedded. The **verbatim table** is what gets returned to the model.

Search is optimised for meaning; answering is optimised for exact digits. Neither is compromised. The same mechanism covers all three content types, because a vision caption plays exactly the same role for a figure.

Narrative text is deliberately **not** described this way. Prose already embeds well, and summarising it would throw away the specific wording a user might quote.

3. Problems found and fixed

All four were found by reading real extraction output, not predicted in advance.

3.1 Tables were indexed twice

`extract_text()` returns the table contents as well, flattened:

```
iPhone $ 40,665 $ 39,570 $ 162,863 $ 153,105 Mac 7,382 8,235 28,669 26,012
```

The columns are gone. That string is searchable, and when retrieved the model sees digits with no headers and guesses. This is the most likely source of wrong numbers in a financial RAG system, and it is invisible unless you look for it.

Fix. Find the tables first, note where they sit on the page, then extract the page text with those areas removed. Each table is stored once, as Markdown.

3.2 Column headers were missing entirely

This was the critical one.

pdfplumber's table area starts at the first **data** row. The period headers sit above it:

Three Months Ended		Nine Months Ended	
June 25, 2022	June 26, 2021	June 25, 2022	June 26, 2021

They were silently dropped, so tables reached the index as grids of unlabelled numbers.

Ask "what were iPhone net sales in Q3 2022?" and that table offers **four** candidate values with no way to choose. The model picks one and is wrong most of the time. No prompt engineering repairs data that is not there.

Fix. Read the strip of page just above each table and attach it. **30 of 30 tables now carry their headers.**

3.3 Tables came out ragged

Financial PDFs put the currency symbol in its own cell so the symbols line up. pdfplumber reports that as a real column, so the page 4 statement extracts as **eleven** columns where five exist.

Deleting columns that are entirely symbols does not work, because the rows disagree about what each column holds:

```
Products : ["Products", "$", "63,355", "$", "63,948", ...]
Services : ["Services", "19,604", "", "17,486", "", ...]
```

Column 1 is \$ in one row and a real value in the next.

Fix. Clean each row on its own, then make every row the same width as the most common one. This realigns the grid as a side effect:

```
["Products", "63,355", "63,948", "245,241", "232,309"]
["Services", "19,604", "17,486", "58,941", "50,148"]
```

Page 4 went from 11 columns to 5. Page 18 went from 11 to 7, exactly matching its 7 headers. A stray % is joined to the number before it, so 3 and % become 3% rather than two cells or a bare 3.

3.4 Narrative crowded out the tables

In this filing the prose *discusses* what the tables *quantify*. For "What were iPhone net sales in Q3 2022?", the sentence "iPhone net sales increased during the third quarter..." is a closer word-for-word match than any table description. So the top results filled with prose, the table never reached the model, and the answer was "iPhone net sales increased" - relevant, cited, and useless.

Fix. Reserve slots in the context for tables. If too few appear, run a second table-only search and add the best ones.

Why not have an LLM decide which type to search. It adds a round-trip to every question, and a wrong decision produces a confident wrong answer with no way to recover. It also cannot handle questions that need both, like "how did iPhone revenue change and why?". Reserving slots has nothing to get wrong, and the worst case is two wasted context slots.

4. Answering

The prompt targets this document's specific failure modes rather than generic accuracy advice.

- 1 **The period trap.** The model is told tables carry both three-month and nine-month figures, told to identify the column before quoting, and given a default (quarterly) that it must state.
- 2 **Calculations must be labelled.** Arithmetic not printed in the filing is allowed, but must show its inputs and be marked as calculated.

- 3 **A fixed refusal sentence.** Given no relevant context, a model that has read many SEC filings will invent a convincing answer. A fixed sentence turns a silent wrong answer into a visible gap, and makes refusals countable.

Every piece of context arrives with a header like `[table on p.10]` that the model is told to repeat, which makes citations checkable. The question is placed **after** the context, because models attend most reliably to the end of a prompt.

5. Storage

Two stores, joined by one id.

Chroma holds one embedding per element, computed from the description. A search returns ids.

SQLite holds the full element, keyed by that id. A search result becomes one `SELECT ... WHERE id IN (...)` for exactly the rows needed.

The first version kept the documents in a JSON file. That was the wrong call. It loaded the whole file into memory at startup, rewrote the entire file to save one element, lost data if two processes wrote at once, and left an unreadable file if anything crashed mid-write. SQLite fixes all four, is in the standard library, and is the same amount of code.

Metadata started as a JSON column and is now nine typed columns. The heterogeneity argument did not survive checking the data: the parser emits exactly nine keys, all simple values, and the set is stable. Real columns mean the database enforces types and `WHERE n_rows > 10` works directly.

The only JSON left is an optional export (`ingest --export-json`) for reading the store by eye. Nothing in the pipeline reads it.

6. Evaluation

20 questions with answers taken directly from the filing, in five categories. Correctness is an exact string check: for financial QA the truth is a specific figure, so *"did 82,959 appear?"* is a complete test that costs nothing and cannot itself hallucinate.

Two things make it harder than a plain accuracy count:

- **Forbidden answers.** Q1 must contain 82,959 and must **not** contain 304,182, the nine-month figure in the next column. Producing both means the system cannot tell the periods apart.
- **A refusal category.** *"How many iPhone units did Apple sell?"* is plausible, on-topic, and absent from any 10-Q. A system that never refuses is not accurate, just agreeable.

Results: 19 / 20

Category	Passed	Total
table	10	10
table_multi	3	3
text	3	4
figure	1	1

Category	Passed	Total
refusal	2	2
overall	19	20

A typical answer:

> "Assuming the three-month (quarterly) period, Apple's total net sales for the > three months ended June 25, 2022 were \$82,959 million [table on p.18]."

The one failure

q16 - "Why did Japan's net sales decrease in the third quarter of 2022?"

> "Japan net sales decreased during the third quarter of 2022 compared to the > same period in 2021 due primarily to lower net sales of iPhone and iPad > [text on p.19]."

Correct but incomplete. The filing gives two reasons: lower iPhone and iPad sales, **and** the weak yen. The model gave the first and stopped.

The right page was retrieved and cited, so this is an answering problem, not a search problem. A prompt rule requiring all stated reasons would fix it. It is left unfixed and reported instead, because one measured failure with a known cause is worth more than a perfect score obtained by adjusting the test.

Two findings worth more than the score

The table failures were false refusals. An earlier run scored 18/20 by declining to answer questions whose data it held. Because every answer keeps a record of what was retrieved, the cause was traceable in three steps: the values were in the store, the right rows were not in the context, and a table-only search put the correct table **third** in both cases - one place below the reserved-slot limit of two. Raising it to four fixed both.

One test was passing for the wrong reason. The figure test expected the word "no" - which is inside "not", so the generic refusal "*the provided document excerpts do **not** contain this information*" satisfied it. The only test covering a required content type was green while never exercising it.

The real problem was that a figure is indexed by what it **shows** ("a black silhouette of the Apple logo"), while the question asks what the document **contains**. Those share no words, and better captioning would not help. Fixed by reserving a figure slot when the question mentions figures, and by requiring the word "logo" in the answer.

7. Assumptions

- 1 The PDF has a real text layer. True here; a scanned filing would need OCR.
- 2 Correctness means agreeing with the document, not with the outside world. A misprint in the filing would count as correct.
- 3 Where a question does not say which period, quarterly is assumed and the assumption is stated in the answer.
- 4 One document, so no cross-document disambiguation.
- 5 English only.

8. Challenges

Challenge	How it was handled
Headers outside the detected table area	Read the strip of page above it (3.2)
Ragged grids from currency symbols	Clean each row, then match the common width (3.3)
Table text duplicated into the narrative	Remove table areas before reading text (3.1)
Financial tables do not embed well	Search a written description instead (2.3)
Narrative crowding out tables	Reserve context slots for tables (3.4)
The document has no figures	Reported, with the evidence, rather than faked
Both chosen models were retired mid-build	Model names live in config, so it was a three-line change
A retry loop hid a permanent error as slowness	Tell transient and permanent errors apart; keep one retry layer
Free-tier limit of 5 requests a minute	Pace the calls; use a lighter model for bulk work

The last three are worth a note. Two of them failed **silently**: ingestion exited successfully with the index built, while a fifth of the tables had quietly fallen back to a weaker description. Nothing looked broken; the system was just worse. Retrying does not help there, because the problem is a sustained rate, not a temporary blip. The producer has to slow down.

9. What would come next

Improvement	Why
Load the numbers into SQL columns	The strongest available upgrade. SELECT cannot misread a column, but a model can. Today correctness still depends on the model reading a grid correctly.
Use the filing's XBRL data	SEC filings ship machine-readable tagged financials, and this one lists them in its own exhibits. In production, take the numbers from XBRL and use RAG only for the narrative.
Measure search separately from answering	Both table failures were search failures, yet only end-to-end accuracy was measured. Labelling which rows <i>should</i> be found would catch that in seconds, with no API calls.
More questions	20 is few enough that one question moves the score by 5 points. Generating 200 from the tables and checking a sample would make the number meaningful.
Counterfactual tests	Change a number in the source and confirm the answer changes. Proves the system is reading the document, not reciting what it already knows about Apple.
Keyword search alongside vector search	Exact line-item names like "Vendor non-trade receivables" are where word matching beats embeddings.
Containerisation	Straightforward; left out for time.

10. Reproducing

```
pip install -r requirements.txt
cp .env.example .env          # add GOOGLE_API_KEY or GEMINI_API_KEY

python -m app.cli ingest      # the filing is already in data/raw/
python -m app.cli inspect
python -m app.cli ask "What were iPhone net sales in Q3 2022?"

python -m tests.test_storage  # offline, no API key
```

```
python -m eval.run_eval          # the 20 questions
```

The reasoning behind each choice, including the options that were rejected, is written into the code next to the code it explains.